

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	 UNIVERSIDAD TECNOLOGICA NACIONAL	Ingeniería y Calidad de Software - 2026
--	---	--

Trabajo Práctico Integrador

Objetivo

Los alumnos deberán desarrollar una aplicación que permita realizar la **estimación, seguimiento y medición de proyectos de software**, aplicando los conceptos y prácticas adquiridos durante la asignatura.

El proyecto deberá desarrollarse obligatoriamente utilizando:

- **Go (Golang)** como lenguaje de programación.
- **Scrum** para la organización y gestión del proyecto.
- **SDD (Specification-Driven Development)** para la especificación de funcionalidades.
- **BDD (Behavior-Driven Development)** para la definición y validación de comportamientos.
- **TDD (Test-Driven Development)** para el desarrollo y prueba de componentes.
- **Git** para control de versiones.
- Herramientas de **Inteligencia Artificial** como soporte al proceso de desarrollo.

Los equipos estarán integrados por un máximo de **5 alumnos**.

Proyecto: Software Metrics & Estimation

Se deberá desarrollar una aplicación que permita administrar un proyecto de software y obtener información relacionada con su **estimación, planificación, seguimiento y calidad**.

La aplicación podrá ser web, de escritorio o consola, pero el núcleo de la solución y sus reglas de negocio deberán estar desarrollados en **Go**.

Requerimientos mínimos

1. Gestión de proyectos

La aplicación deberá permitir:

- Crear y modificar proyectos.
- Registrar integrantes.

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	 UNIVERSIDAD TECNOLOGICA NACIONAL	Ingeniería y Calidad de Software - 2026
---	--	--

- Registrar fecha de inicio y finalización.
- Consultar el estado de un proyecto.

2. Product Backlog

Para cada proyecto se deberá administrar un Product Backlog.

Cada elemento deberá contener como mínimo:

- Identificador.
- Título.
- Descripción.
- Prioridad.
- Estado.
- Story Points.
- Criterios de aceptación.

3. Gestión de Sprints

La aplicación deberá permitir:

- Crear Sprints.
- Definir un Sprint Goal.
- Asignar historias a un Sprint.
- Registrar historias completadas.
- Cerrar un Sprint.
- Consultar Sprints anteriores.

4. Estimación

La aplicación deberá permitir estimar historias mediante **Story Points**.

Además, deberá implementar un mecanismo de **Planning Poker** que permita:

- Registrar las estimaciones individuales.
- Mantenerlas ocultas hasta finalizar la votación.
- Mostrar las estimaciones realizadas.
- Detectar diferencias entre estimaciones.
- Realizar nuevas rondas.
- Registrar la estimación acordada.

5. Registro de esfuerzo

Para cada historia o tarea se deberá poder registrar:

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	 UNIVERSIDAD TECNOLOGICA NACIONAL	Ingeniería y Calidad de Software - 2026
---	---	--

- Integrante.
- Fecha.
- Actividad realizada.
- Horas trabajadas.

Esto deberá permitir comparar posteriormente el esfuerzo estimado con el esfuerzo real.

6. Gestión de defectos

El sistema deberá permitir registrar defectos indicando como mínimo:

- Descripción.
- Severidad.
- Estado.
- Historia relacionada.
- Sprint de detección.
- Sprint de resolución.

7. Métricas

La aplicación deberá calcular como mínimo:

- Story Points planificados.
- Story Points completados.
- Velocidad del equipo.
- Horas estimadas.
- Horas reales.
- Desviación entre esfuerzo estimado y real.
- Porcentaje de historias completadas.
- Cantidad de defectos detectados.
- Cantidad de defectos resueltos.

8. Dashboard

El sistema deberá presentar un Dashboard con información sobre el estado del proyecto y representaciones gráficas de algunas de las métricas obtenidas.

9. Reportes

La aplicación deberá generar un reporte de un proyecto o Sprint incluyendo:

- Historias planificadas y completadas.
- Estimaciones.
- Esfuerzo registrado.
- Métricas.

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	 UNIVERSIDAD TECNOLOGICA NACIONAL	Ingeniería y Calidad de Software - 2026
--	---	--

- Defectos.
-

Metodología de desarrollo

El proyecto deberá gestionarse mediante **Scrum**, utilizando los siguientes roles:

- **Product Architect:** profesores.
- **Agile Enabler:** un integrante del equipo.
- **Product Builders:** integrantes del equipo responsables de construir el producto.

El proyecto deberá organizarse mediante un **Product Backlog y Sprints**, realizando las correspondientes actividades de planificación, seguimiento, revisión y retrospectiva.

Aplicación de SDD

Las funcionalidades principales deberán ser especificadas **antes de su implementación**.

Las especificaciones deberán establecer como mínimo:

- Objetivo.
- Entradas.
- Salidas esperadas.
- Reglas de negocio.
- Restricciones.
- Casos límite.
- Condiciones de error.
- Criterios de aceptación.

Las especificaciones deberán mantenerse versionadas junto con el proyecto.

Aplicación de BDD

Las funcionalidades seleccionadas deberán contar con escenarios que describan su comportamiento mediante:

Given – When – Then

Los escenarios deberán contemplar:

- Casos normales.
- Casos alternativos.
- Casos límite.
- Errores.

Siempre que sea posible deberán automatizarse.

Aplicación de TDD

Las principales reglas de negocio y cálculos deberán desarrollarse utilizando el ciclo:

RED → GREEN → REFACTOR

Se deberán implementar pruebas unitarias en Go para, como mínimo:

- Cálculo de métricas.
- Cálculos de estimación.
- Reglas de negocio.
- Validaciones.

El equipo deberá mantener evidencia del proceso de TDD mediante el historial del repositorio.

Uso de Inteligencia Artificial

Se aceptará el uso de herramientas de Inteligencia Artificial durante el proyecto para tareas como:

- Análisis de requisitos.
- Elaboración y revisión de especificaciones.
- Generación de código.
- Generación de pruebas.
- Refactorización.
- Revisión de código.

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	 UNIVERSIDAD TECNOLOGICA NACIONAL	Ingeniería y Calidad de Software - 2026
---	---	--

- Documentación.

Todo resultado generado mediante IA deberá ser **comprendido, revisado y validado por el equipo**.

Los alumnos serán responsables por todo el código incorporado al proyecto, independientemente de que haya sido desarrollado manualmente o generado con asistencia de IA.

Plan de trabajo

Se propone desarrollar el proyecto mediante:

- Sprint 0: La Preparación
 - Objetivo: Formar equipos, asignar roles, configurar el entorno de trabajo.
 - Tareas:
 - Crear el repositorio en GitHub/GitLab.
 - Configurar el tablero del proyecto en GitHub Projects.
 - Reunión inicial: El profesor presenta la visión del producto.
 - El equipo crea el Product Backlog inicial, escribiendo las primeras historias de usuario junto al Cliente.
 - Sprint 1: El MVP
 - Objetivo: Entregar una versión funcional básica.
 - Ceremonias: Se realizan todas: Planning, Daily, Review y Retrospective.
 - Sprint 2: La Interfaz Objetivo: Dotar al software de una interfaz usable.
 - Sprint 3: Funcionalidad y Calidad
 - Objetivo: Añadir funcionalidades clave y robustecer el sistema. (Implementar la visualización)
 - Sprint 4: El Cierre y la Entrega Final
 - Objetivo: Pulir el producto, documentar y preparar la entrega final (El sistema permitirá exportar un informe del proyecto a un archivo en formato PDF)
 - Sprint Review Final: Presentación formal del producto completo al Cliente.
-

Trazabilidad

El equipo deberá demostrar trazabilidad entre:

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	 UNIVERSIDAD TECNOLOGICA NACIONAL	Ingeniería y Calidad de Software - 2026
---	---	--

Historia de Usuario → Especificación SDD → Criterios de Aceptación → Escenarios BDD → Tests → Código Go

Durante la presentación final se seleccionará al menos una funcionalidad para demostrar este recorrido completo.

Entregables

Cada equipo deberá entregar:

1. Repositorio Git con historial de contribuciones.
 2. Tablero Scrum.
 3. Product Backlog y Sprint Backlogs.
 4. Especificaciones SDD.
 5. Escenarios BDD.
 6. Pruebas automatizadas.
 7. Código fuente en Go.
 8. Evidencias de aplicación de TDD.
 9. Software funcional.
 10. Informe de métricas y cobertura de pruebas.
 11. Actas de retrospectivas.
 12. Documentación técnica y manual breve de usuario.
 13. Presentación y demostración final.
-

Criterios de evaluación

Producto funcional – 25 %

- Cumplimiento de los requerimientos.
- Correctitud.
- Usabilidad.
- Robustez.

SDD, BDD y TDD – 25 %

- Calidad de las especificaciones.
- Calidad de escenarios BDD.

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	 UNIVERSIDAD TECNOLOGICA NACIONAL	Ingeniería y Calidad de Software - 2026
---	---	--

- Aplicación de TDD.
- Pruebas automatizadas.
- Trazabilidad.

Calidad del software – 20 %

- Arquitectura.
- Calidad del código Go.
- Modularidad y mantenibilidad.
- Pruebas y cobertura.
- Manejo adecuado de errores.

Gestión del proyecto – 20 %

- Calidad del Product Backlog.
- Planificación y cumplimiento de Sprints.
- Gestión del tablero.
- Reviews y retrospectivas.
- Uso adecuado del repositorio Git.

Trabajo en equipo y presentación – 10 %

- Participación de los integrantes.
- Colaboración.
- Capacidad para justificar las decisiones tomadas.
- Calidad de la presentación y demostración final.